

Extended Abstract

Motivation Fine-tuning large language models (LLMs) with reinforcement learning is expensive due to the need for on-policy sampling. Instruction-following and math reasoning tasks (e.g. Countdown) require many queries to converge. The goal of this project is to improve sample efficiency by incorporating off-policy updates into an RLOO-style method. In particular, our approach focuses on tasks that involve a deep level of systematic reasoning and require factual accuracy, in order to evaluate the effects of more sample-efficient methods on LLM capabilities to maintain consistent reasoning skills.

Method We focus our project on the Countdown task. Toward this end, we start with a vanilla RLOO implementation, using the WarmStart dataset and TinyZero for fine-tuning. Our proposed extension incorporates off-policy sampling in order to reduce the number of on-policy samples needed for convergence, by using a replay buffer.

Implementation The backbone of this project is the default SFT and RLOO implementations outlined in the default project, trained on the WarmStart dataset and TinyZero. Our proposed extension extends REINFORCE-leave-one-out to incorporate off-policy sampling and a replay buffer in order to enable multiple updates per batch, using fewer on-policy samples. Due to memory constraints, we were only able to run RLOO with a batch size of 1 or 2, and a K-value of 24.

Results Due to time and compute constraints, we did not achieve a working RLOO + off-policy sampling model in time for the project deadline. Our initial RLOO implementation achieved a score of 0.1063 on the leaderboard using the held-out prompts, and our initial SFT model achieved a score of 0.2350.

Discussion One danger of off-policy sampling is instability during training—since the off-policy samples may not be well-aligned with the target, the additional policy updates may cause unwanted noise. One mitigating method for this is to weigh these samples less than on-policy samples in order to bias the model toward a relevant goal. The primary problem we faced during implementation was a lack of growth from the base RLOO method over SFT—since our proposed extension is built on RLOO as a baseline, we were limited by little success on the initial implementation.

Conclusion We ultimately did not achieve the goal of this project, which was to train a model that performs similarly to our RLOO implementation using far fewer on-policy samples.

Off-Policy Sampling for Fine-Tuning LLMs

Andrew Nordhagen

Department of Computer Science
Stanford University
aenord@stanford.edu

Abstract

Fine-tuning large language models with reinforcement learning is expensive due to the need for on-policy sampling. Instruction-following and math reasoning tasks (e.g. Countdown) require many queries to converge. The goal of this project is to improve sample efficiency by incorporating off-policy updates into an RLOO-style method. Toward this end, we start with a vanilla RLOO implementation, using the WarmStart dataset (2) and TinyZero Prompts dataset (3) for fine-tuning, and propose an extended version that incorporates a replay buffer and off-policy sampling in order to achieve similar performance with fewer on-policy samples.

1 Introduction

Fine-tuning large language models (LLMs) with reinforcement learning is expensive due to the need for on-policy sampling. Instruction-following and math reasoning tasks require many queries to converge. The goal of this project is to improve sample efficiency by incorporating off-policy updates into an RLOO-style method. Toward this end, we start with a vanilla RLOO implementation, using the WarmStart dataset and TinyZero for fine-tuning, and propose an extended version that incorporates a replay buffer and off-policy sampling in order to achieve similar performance with fewer on-policy samples. The task we focus on to test our implementations of SFT, RLOO, and off-policy sampling is math reasoning, as described in the Countdown task. The goal of evaluation is to measure the impact of off-policy sampling on mathematical reasoning capabilities of an LLM, and the potential tradeoffs in stability introduced by improving sample efficiency.

2 Related Work

Countdown Task The Countdown task (4) is a math reasoning benchmark designed to test the arithmetic and planning capabilities of LLMs. The problem defines a target number and a set of integers, and requires the model to generate an equation using each of the provided numbers (only once each) as well as basic arithmetic operators (+, -, *, /) to compute the target number. This task is challenging in the natural language space due to the combinatorial nature of the problem and the need for symbolic reasoning, which makes it an appealing challenge for evaluating the reasoning capabilities of LLMs and the efficacy of RL fine-tuning.

REINFORCE Leave-One-Out (RLOO) RLOO (1) is a variant of REINFORCE which estimates a baseline using leave-one-out averaging to reduce variance in the policy gradient. It has been shown to be effective in small-batch or few-sample settings for LLM finetuning (7); however RLOO uses on-policy sampling which can be expensive, motivating the possibility of an augmented version with off-policy sampling.

Off-Policy Sampling for RLHF Recent work in post-training LLMs has focused on combining on-policy and off-policy learning. Tang et al. (6) propose a hybrid algorithm that can leverage static datasets in a reinforcement learning framework to improve sample efficiency. Other approaches,

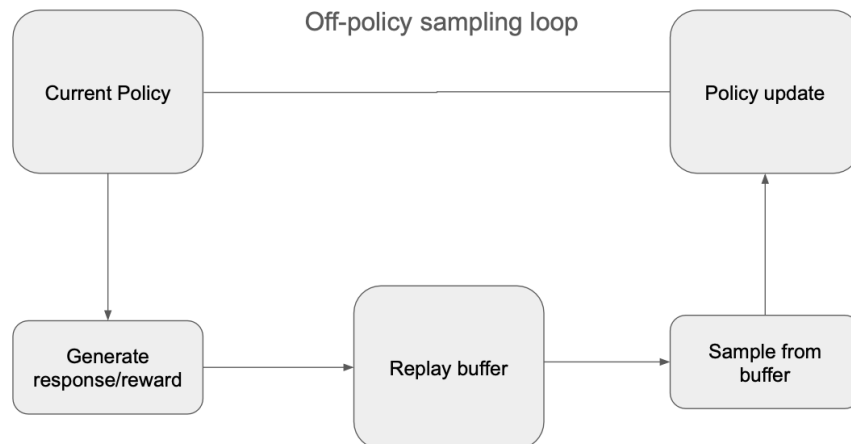


Figure 1: Proposed Method Overview.

such as Tapered Off-Policy REINFORCE (7), explore weighted sampling and modified baselines to safely incorporate off-policy data. Our proposed method extends these efforts to incorporate off policy sampling, by way of a replay buffer that allows for multiple policy updates per iteration using previous prompt/response/reward pairs.

3 Method

We focused our project on the Countdown task. Toward this end, we start with a vanilla RLOO implementation, using the WarmStart dataset and TinyZero Prompts dataset for fine-tuning. The prompt structure used for both SFT and RLOO was as follows:

"A conversation between User and Assistant. The user asks a question, and the Assistant solves it. " "The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. " "User: Using the numbers [numbers], create an equation that equals target. You can use basic arithmetic operations (+, -, *, /) and each number can only be used once. " "Show your work in <think> </think> tags. And return the final answer in <answer> </answer> tags, for example <answer> (1 + 2) / 3 </answer>. "

3.1 Datasets

WarmStart: Used for verifier-based scoring, helps guide the model toward structured reasoning strategies in the Countdown task. We trained our initial SFT implementation on this dataset.

TinyZero Prompts: Used to sample prompts for reinforcement learning on math reasoning (Countdown) tasks. This dataset was used with RLOO to further fine-tune the model.

4 Experimental Setup

SFT: We trained SFT on the WarmStart dataset for 3 epochs, with a batch size of 4 to accommodate memory constraints. We used the AdamW optimizer with a learning rate of $5e-5$, and initialized the model from the Qwen 2.5 0.5B Base model via Huggingface.

RLOO: Our RLOO model was initialized from the SFT checkpoint from the previous step. We trained this method on the TinyZero Prompts dataset for 2 epoch, with $K=6$ and a batch size of 1 to accommodate memory constraints. We estimate that barring compute costs, this would have trained much faster and achieved much better results with a larger batch size and K value, as well as longer sequence lengths.

RLOO + off-policy sampling: We did not finish implementing and training the proposed extension to RLOO. However, in order to evaluate this method against RLOO, we would have initialized it from the same SFT checkpoint and trained with the same hyperparameters, in order to isolate the effects of off-policy sampling in the RLOO framework.

5 Results

5.1 Quantitative Evaluation

Our initial RLOO implementation achieved a score of 0.1063 on the leaderboard using the held-out prompts, and our initial SFT model achieved a score of 0.2350.

Quantitatively, we did not achieve meaningful results for the proposed RLOO + off-policy sampling method, since the initial RLOO method that it depends on was not fully implemented and trained to success. We ran into memory bottlenecks that significantly extended training time, as well as problems with AWS Spot instances—training was frequently interrupted by the remote machine stopping unexpectedly.

Despite this, we still hypothesize that the proposed off-policy RLOO would reduce sample costs for the Countdown task. A potential risk is that off-policy updates may introduce instability if the re-sampled goal is poorly related to the current goal, and while adding importance weighting to this method may mitigate this effect, tuning those weights would likely be difficult.

5.2 Qualitative Analysis

After fine-tuning as described above, the RLOO model still failed to consistently generate correct solutions. Over the course of training, the generated output more consistently matched the desired structure, with the answer encased in <answer> tags for simpler parsing, but often hallucinated input numbers or output incorrect equations that did not evaluate to the correct target. Additionally, the result output was occasionally truncated before the model reached an answer, resulting in no score—this problem would be solved with a larger memory budget, since we reduced the maximum length of each output to avoid OOM errors.

6 Discussion

One danger of off-policy sampling is instability during training—since the off-policy samples may not be well-aligned with the target, the additional policy updates may cause unwanted noise. One mitigating method for this is to weigh these samples less than on-policy samples in order to bias the model toward a relevant goal. The primary problem we faced during implementation was a lack of growth from the base RLOO method over SFT—since our proposed extension is built on RLOO as a baseline, we were limited by little success on the initial implementation.

7 Conclusion

We ultimately did not achieve the goal of this project, which was to train a model that performs similarly to our RLOO implementation using far fewer on-policy samples. We faced challenges in implementing the initial methods (SFT and RLOO); primarily, we were unable to obtain access to a GPU instance with enough memory to sustain larger batch sizes or longer sequences, which bottlenecked our training and throttled the learning of both methods, both in terms of speed and efficacy at the given task.

8 Team Contributions

- **Andrew Nordhagen:** This was a solo project—I did the entirety of the work.

Changes from Proposal As the default project guidelines were relaxed, I only focused on SFT + RLOO for the Countdown task, and did not implement DPO for UltraFeedback.

References

- [1] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms, 2024. URL <https://arxiv.org/abs/2402.14740>.
- [2] Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective stars, 2025. URL <https://arxiv.org/abs/2503.01307>.
- [3] Jiayi Pan, Junjie Zhang, Xingyao Wang, Lifan Yuan, Hao Peng, and Alane Suhr. Tinyzero. <https://github.com/Jiayi-Pan/TinyZero>, 2025.
- [4] Kanishk Gandhi, Denise Lee, Gabriel Grand, Muxin Liu, Winson Cheng, Archit Sharma, and Noah D. Goodman. Stream of search (sos): Learning to search in language, 2024. URL <https://arxiv.org/abs/2404.03683>.
- [5] Brian R. Bartoldson, Siddarth Venkatraman, James Diffenderfer, Moksh Jain, Tal Ben-Nun, Seanie Lee, Minsu Kim, Johan Obando-Ceron, Yoshua Bengio, and Bhavya Kailkhura. *Trajectory balance with asynchrony: Decoupling exploration and learning for fast, scalable LLM post-training*, 2025.
- [6] Yunhao Tang, Taco Cohen, David W. Zhang, Michal Valko, and Rémi Munos. *RL-Finetuning LLMs from on- and off-policy data with a single algorithm*, 2025.
- [7] Nicolas Le Roux, Marc G. Bellemare, Jonathan Lebensold, Arnaud Bergeron, Joshua Greaves, Alex Fréchette, Carolyne Pelletier, Eric Thibodeau-Laufer, Sándor Toth, and Sam Work. *Tapered off-policy reinforce: Stable and efficient reinforcement learning for LLMs*, 2025.